



01211373 Machine Learning and Programming for Industry

DEEP LEARNING

Convolutional Neural Networks

Lecture 9 — Automated Visual Inspection

Last Week → This Week

WEEK 8 RECAP

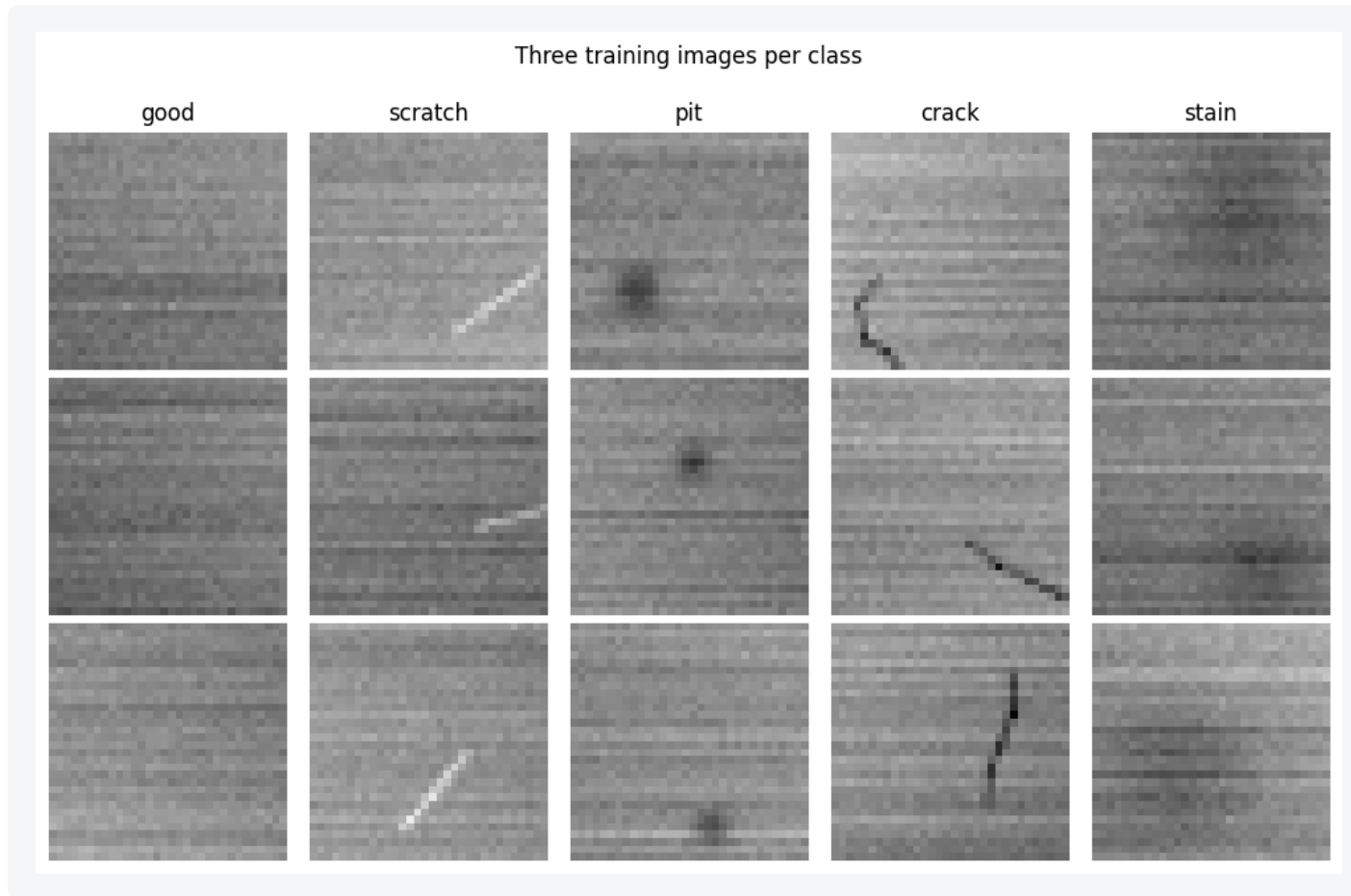
- Custom modules built from a reusable block
- The loss function encodes what “wrong” means
- Early stopping with a three-way split
- Ship weights + architecture + scaler

TODAY

- The first input with no hand-made features: raw pixels
- Why flattening an image destroys what matters
- Convolution, weight sharing and pooling
- Batch normalization and dropout
- Seeing what the network learned

Today's Problem: Surface Inspection

A camera photographs every brushed-steel part on the conveyor. Five categories; defects can appear anywhere in the frame.



scratch

thin, bright, straight

pit

small, dark, round

crack

jagged, dark, branching

stain

large, faint, diffuse

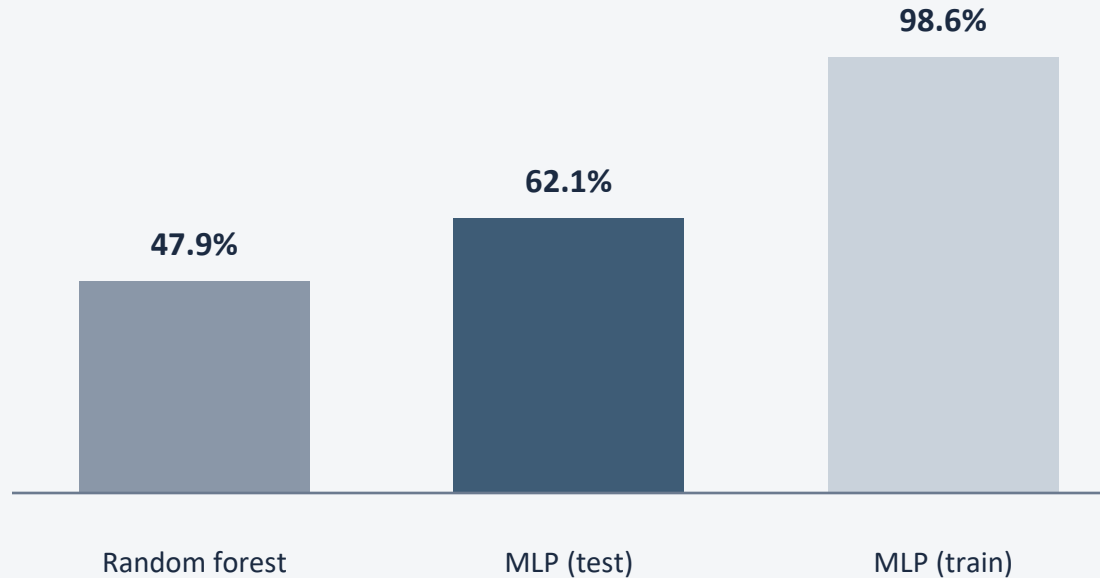
No features this time — just 1,024 pixel values.

Why Flattening an Image Fails



Flatten a 32×32 image and you get 1,024 unrelated columns. Pixel (5, 7) and its neighbour (5, 8) are no more related than pixels at opposite corners.

Tools you already know, on raw pixels (test accuracy)



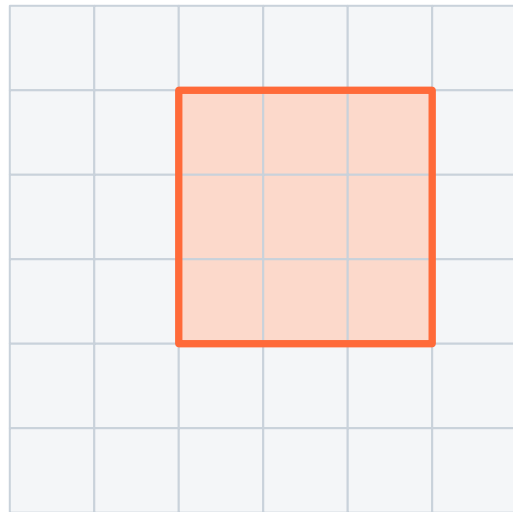
Look at the MLP's two bars. 98.6% on training images, 62.1% on new ones.

It memorized which pixel positions happened to be dark or bright. A scratch a few pixels to the left activates entirely different input weights — to the MLP, that is a pattern it has never seen.

The Convolution Operation



A small filter — here 3×3 — slides across the image. At each position it multiplies the pixels under it by its weights and sums the result into one output value.

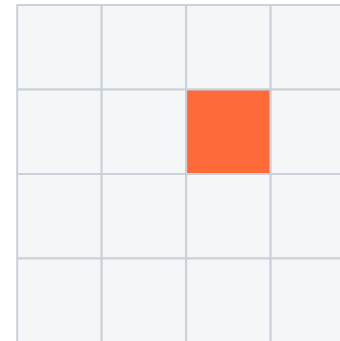


input image

×

-1	0	+1
-1	0	+1
-1	0	+1

3×3 filter



feature map

This filter **is a vertical-edge detector**: large output where brightness changes left-to-right, near zero on a flat surface.

In a CNN nobody chooses these numbers. They are weights, learned by backpropagation exactly like Week 7's.

Output size: a 6×6 input and a 3×3 filter give a 4×4 map. Padding the border by one pixel keeps the output at 6×6 — which is what the lab's padding=1 does.

Two Ideas That Make CNNs Work



Local connectivity

Each output looks at a small neighbourhood, not the whole image. Nearby pixels are treated as related — the structure flattening threw away.



Weight sharing

The same filter is applied at every position. A scratch detector learned in one corner works unchanged in every other corner.

The consequence, measured in the lab

279,173

MLP parameters

23,845

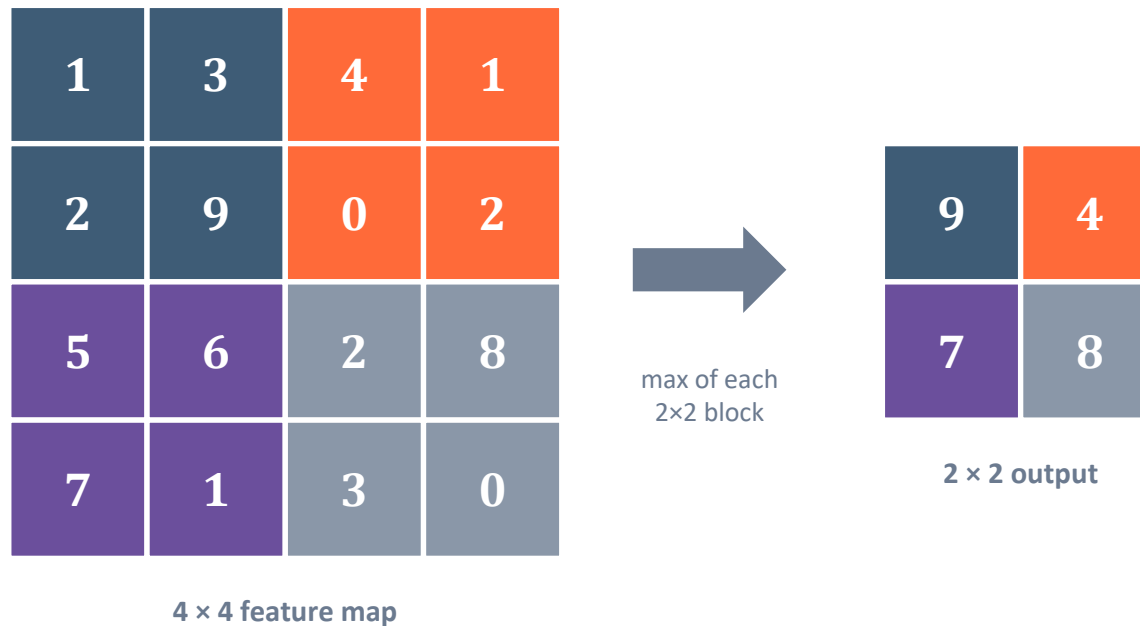
CNN parameters

Twelve times fewer parameters — and far better generalization. Building the right assumption into the architecture beats giving the model more freedom.

Pooling: Keeping What, Dropping Exactly Where



Max pooling keeps the strongest response in each 2×2 region. The map halves in each direction, and small shifts of a feature no longer change the output.

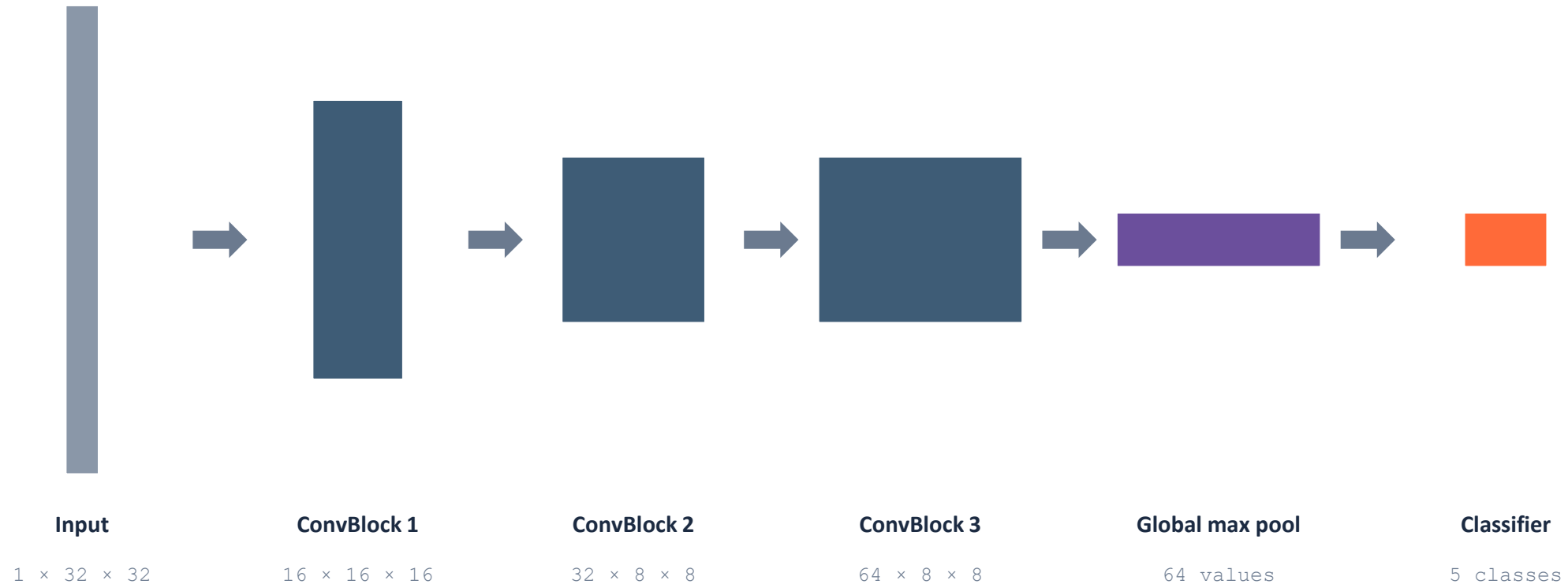


Global max pooling takes this to the limit: one number per channel for the whole image.

The lab's last pooling layer asks one question per filter: **“did this pattern appear anywhere?”** For inspection, that is exactly the right question — a crack is a crack wherever it is.

The Lab's Architecture: Trading Where for What

Spatial size shrinks while the number of channels grows. By the end the network no longer knows where anything is — only what is present.



Each ConvBlock = convolution → batch norm → ReLU → 2×2 max pool (the third block skips the pool)

Batch Normalization & Dropout

Batch normalization

Re-centres and re-scales each channel's activations using the statistics of the current mini-batch. Training becomes faster and far less sensitive to the learning rate — it is Week 2's StandardScaler, applied inside the network, between layers.

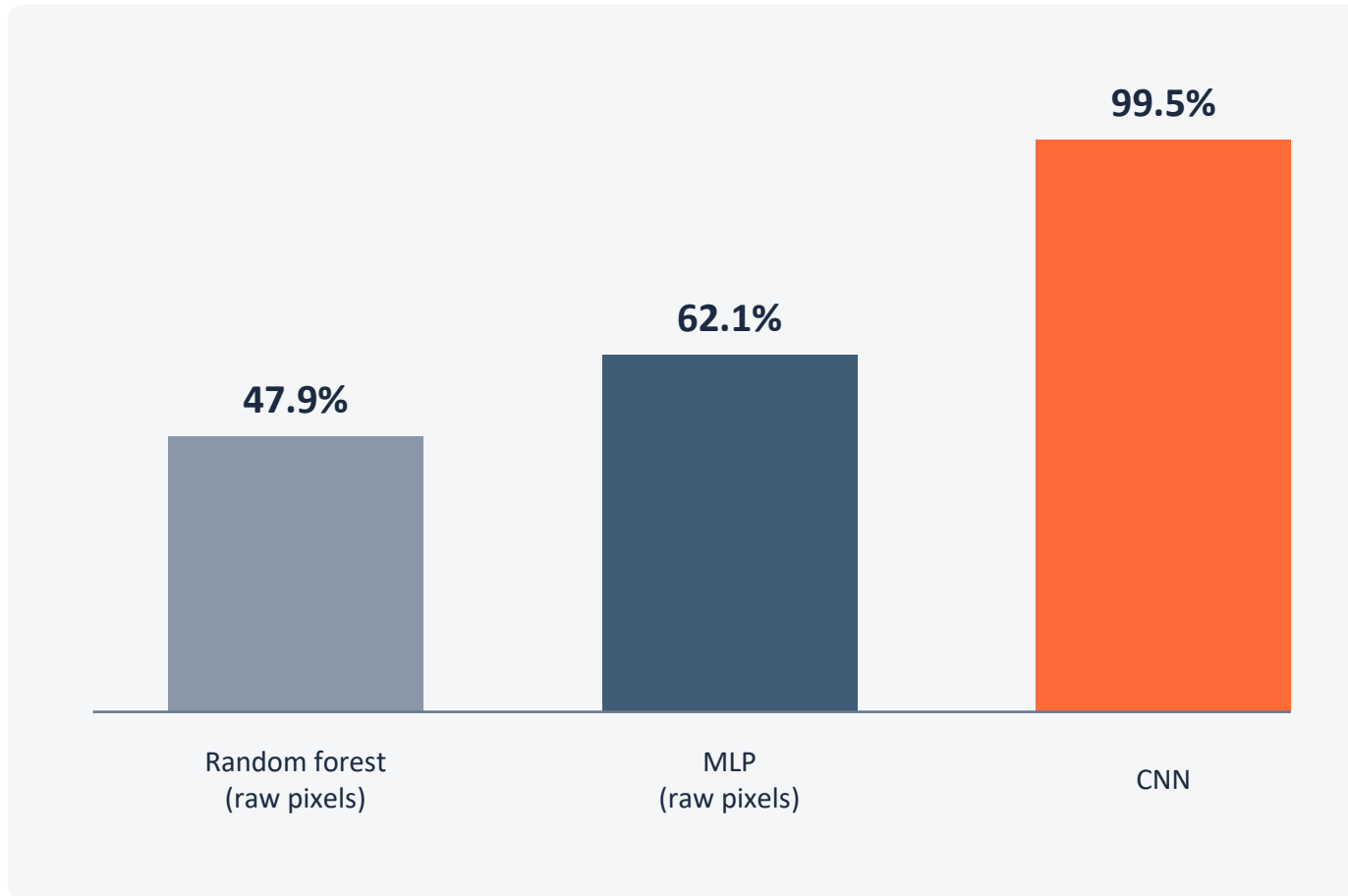
Dropout

During training, randomly zeroes a fraction of activations (30% in the lab) on every step. No single unit can be relied on, so the network learns redundant, more general features. It is regularization, like Week 3's Ridge, by a different route.

Both behave differently in training and in use. `model.train()` turns dropout on and makes batch norm use batch statistics; `model.eval()` turns dropout off and uses the running averages. Forget `model.eval()` before inference and a deployed inspection system gives different answers for the same image.

The Result

Same images, same split, same training budget. Test accuracy on 750 held-out parts:



4 of 750

test images misclassified by the CNN

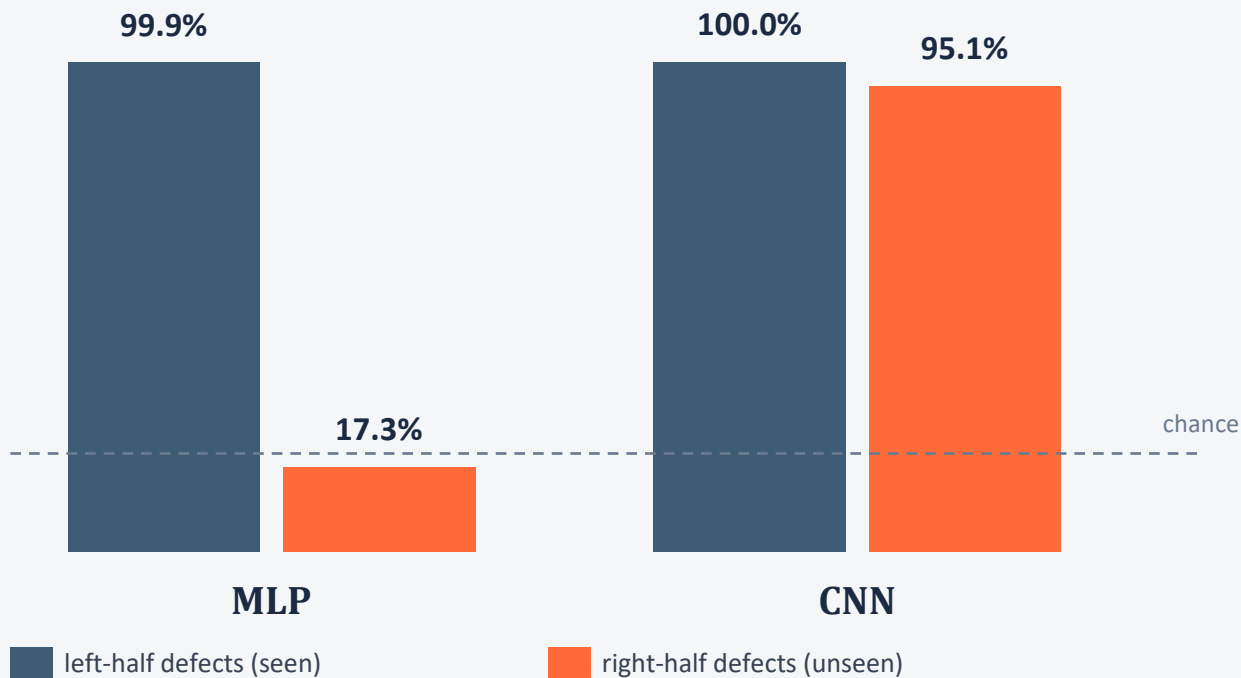
Recall Week 7, where a neural network merely tied a random forest on tabular data. The difference here is the input: raw pixels, where learning the features is the whole problem.

That is the regime deep learning was built for.

The Camera Moved



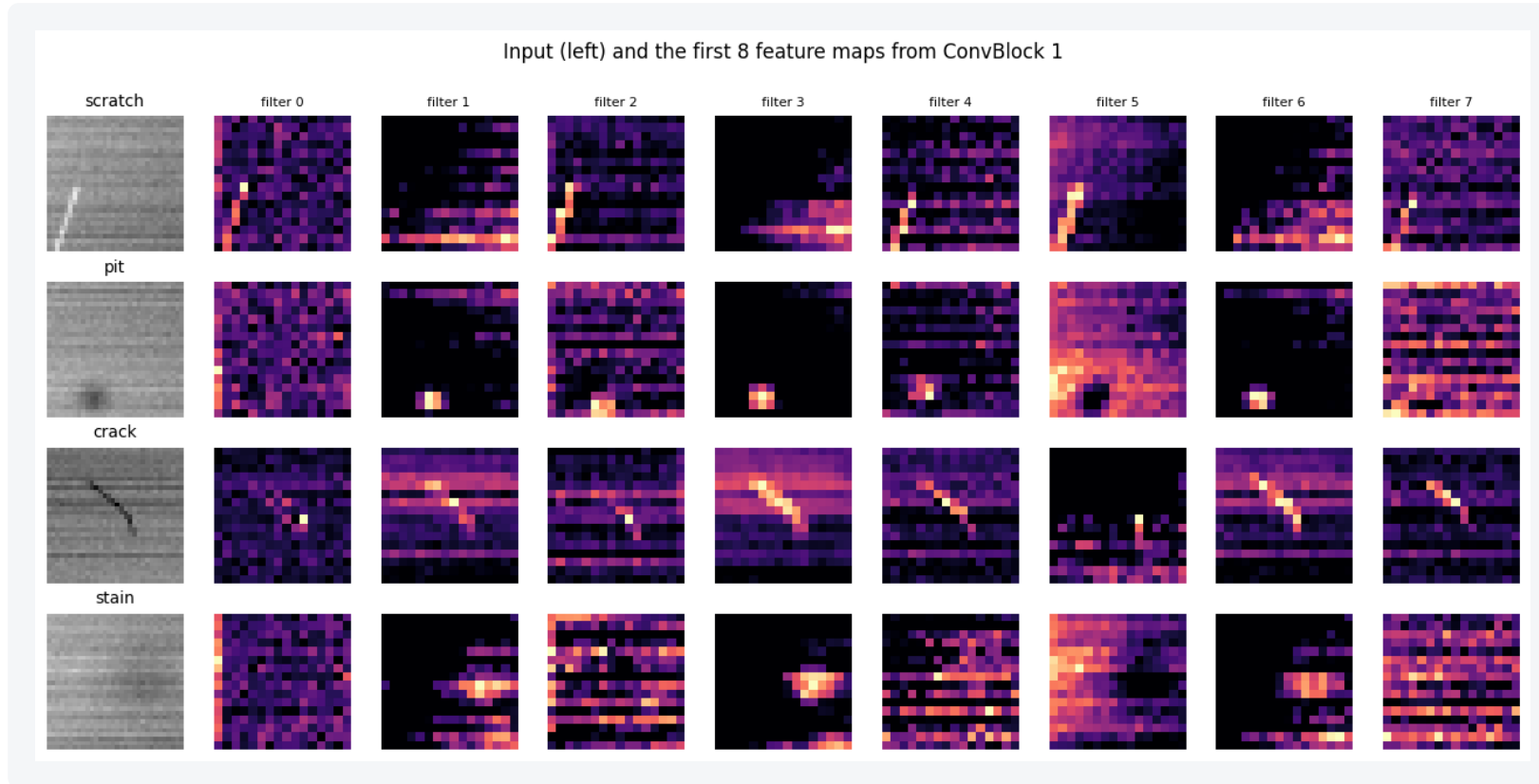
Train with every defect on the left half of the frame — say, because all the training images came from one fixture. Then test on parts where the defects appear on the right half.



The MLP falls below chance. Its right-side input weights never saw a defect during training, so they have nothing to offer.

The CNN barely notices. Its filters slide over the whole frame with the same weights — a crack detector learned on the left already works on the right.

What Did It Learn? (The Lab's Actual Network)



Filters 1, 3 and 6 became dark-spot detectors: they light up on the pit, crack and stain and stay quiet elsewhere.

Filter 5 responds to the bright scratch.

Filters 2 and 4 track the brushed streaks — the background texture the later layers must learn to ignore.

Nobody designed these detectors. They emerged from gradient descent because they reduced the loss. Your own network will learn a different set in a different order — the principle, not the filter numbering, is what carries over.

Taking It to a Real Production Line



The lab uses clean synthetic images and plenty of examples of each defect. Real inspection data is rarely so generous. Two standard remedies:

Data augmentation

Generate extra training images by flipping, rotating, shifting, and varying brightness. It teaches the invariances you need — and would have rescued the Part F failure even for a weaker model.

Transfer learning

Start from a network already trained on millions of general images and fine-tune only its last layers on your parts. Often reaches good accuracy from a few hundred labeled examples instead of thousands.

And fix the camera before fixing the model. Controlled, uniform lighting often does more for accuracy than any architecture change — the stain class in the lab is hard partly because the lighting already varies across the frame.

Today's Lab: Inspect, Then Look Inside

1

Baselines on raw pixels

Random forest and MLP — watch the MLP memorize.

2

Build a CNN from ConvBlocks

Trace the shapes: where shrinks, what grows.

3

Move the camera

Train on left-half defects, test on the right.

4

Look inside

Plot the first layer's feature maps for each defect type.

Key Takeaways



Flattening an image discards neighbourhood structure; convolution keeps it.



Weight sharing means a detector learned in one place works everywhere.



Fewer, well-placed parameters beat more, unconstrained ones.

NEXT WEEK

Recurrent Networks & Sequential Data

LSTMs for forecasting tool wear — where the order of the readings is the signal.